

Part B - Production Architecture

Turning the Amazon-checkout prototype into something a real testing platform could run on - the part of the stack a team leans on to decide what is actually safe to ship.

The architecture is built around two things that are painful to change later: the agent harness, where agents live, and the evaluation loop, where you prove the system is improving instead of quietly rotting.

The prototype already shows the shape of the product: a browser agent can explore a real page, but a graph and validator have to decide what that exploration actually proved. Part B takes that prototype loop and asks what must exist around it before a company could trust it in CI.

I would describe the production system in one sentence: plan the next experiment, act on the browser, capture evidence, validate independently, write memory, and then use that memory to decide what to test next. The rest of the architecture is mostly about making that loop durable, safe, measurable, and cheap enough to run repeatedly.

0. The one belief this rests on

A browser agent and a knowledge graph are good at almost opposite things. The browser agent is opportunistic: it can work out which control adds an item to cart after the DOM changes, but it only exercises the paths it happens to walk and can report success on an action that quietly did nothing.

The graph is the part that remembers: every state seen, every control, what was proved versus merely noticed, and what should exist but has not been found. The crawler discovers; the graph remembers and cross-examines; the harness stops the loop from lying to itself.

This is also the mental model that keeps the design from becoming too magical. I am not asking an LLM to be the source of truth. I am using the LLM where it is useful - operating messy UI and proposing possibilities - and then putting deterministic checks, graph queries, and audit trails underneath it.

1. The multi-agent network

The work is split into small agents with hard boundaries rather than one clever mega-agent. Each agent owns one kind of decision, and the hand-off is a typed contract rather than a shared blob of state.

The table below is a responsibility map, not a demand that every row become a separate deployed service on day one. In an early version, several of these can live in the same process. What matters is that the boundaries are real: the crawler should not validate itself, and the model should not directly write graph truth.

Agent	Owns	LLM?	Boundary enforced
Planner	Frontier, priorities, budgets	No	Decides what to explore; never touches the browser.
Crawler	One UI action via browser-use / Playwright	Small / fast	Executes how; never writes graph truth or decides pass/fail.
Observer	Snapshot: DOM, screenshot, URL, controls	Mostly no	Returns raw evidence, not a trusted summary.
Reasoner	Missed, absent, causal scenarios	LLM proposes; Cypher decides	Deterministic graph queries remain the trustworthy floor.
Validator	Independent post-conditions	No	Nothing is validated until evidence is rechecked.
Healer	Selector repair	Small model + graph	Suggests fixes; Validator confirms.
Triage	Failure clustering and flake calls	LLM + graph history	Escalates; does not silently suppress.
Safety supervisor	Pre-action guardrails	No	Vetoes before the browser sees the task.

Agent	Owns	LLM?	Boundary enforced
Ingestor	Idempotent graph writes	No	Typed, replayable writes only; no model calls.

The most important boundary is between Crawler and Validator. When the Crawler says it changed the quantity, the Validator reads the cart again and checks that the number actually moved.

That boundary is what makes the system trustworthy. A browser agent can be creative and occasionally wrong; the validator is deliberately boring and skeptical. The ingestor then writes only the checked result, not the agent's story about the result.

- **Fails:** retry with backoff up to a budget, then mark the step failed and continue. A failed step is data, not a crash.
- **Stalls:** an action ledger and no-repeat guard turn repeated clicks into a fast veto.
- **Low confidence:** escalate, hand off to a human, or write an explicit unverified marker. Uncertainty is a first-class output.

1.1 The contract between agents

Every hand-off carries intent, expected state, confidence, evidence to seek, risk class, and escalation signal. Those Pydantic-style contracts are the durable interface that lets models or frameworks change underneath.

For example, the planner should not send the crawler a vague instruction like 'test the cart.' It should send a bounded intent: try Change Quantity on the cart page, look for quantity/subtotal evidence, and treat payment or account changes as forbidden. That kind of contract gives the validator something concrete to check.

1.2 How the Validator handles unknown actions

The Validator can only fully confirm an action when it knows what evidence to check. So every action lands in one of three states: done and confirmed, done but unverified, or blocked/failed. The system keeps useful unknown actions, but it does not call them successful until there is proof.

This is important because real applications always have actions that the contract did not anticipate. The system should still record that the action exists, and it may even use it later, but it should not inflate coverage until there is a reliable post-condition. Unknown is acceptable; falsely validated is not.

Validator fallback	How it keeps the result honest
Graph expectation	Read cause-and-effect rules such as change_quantity SHOULD_CAUSE subtotal_change.
Always-true invariants	Check facts that should never break, such as totals matching item sums.
Before / after comparison	Look for meaningful state change when an action should have produced one.
Calibrated judge	Use a different model only when no rule or invariant exists, then measure judge agreement against humans.

The fallback order is intentionally conservative. A known graph expectation is better than a model judge. An invariant is better than a guess. A before/after comparison is useful when the expected state change is visible. The judge is reserved for cases where no deterministic rule exists, and even then its agreement has to be measured over time.

The safe default is simple: admit uncertainty. A tool that tells teams what is safe to ship should prefer 'unverified' over a confident but unsupported pass.

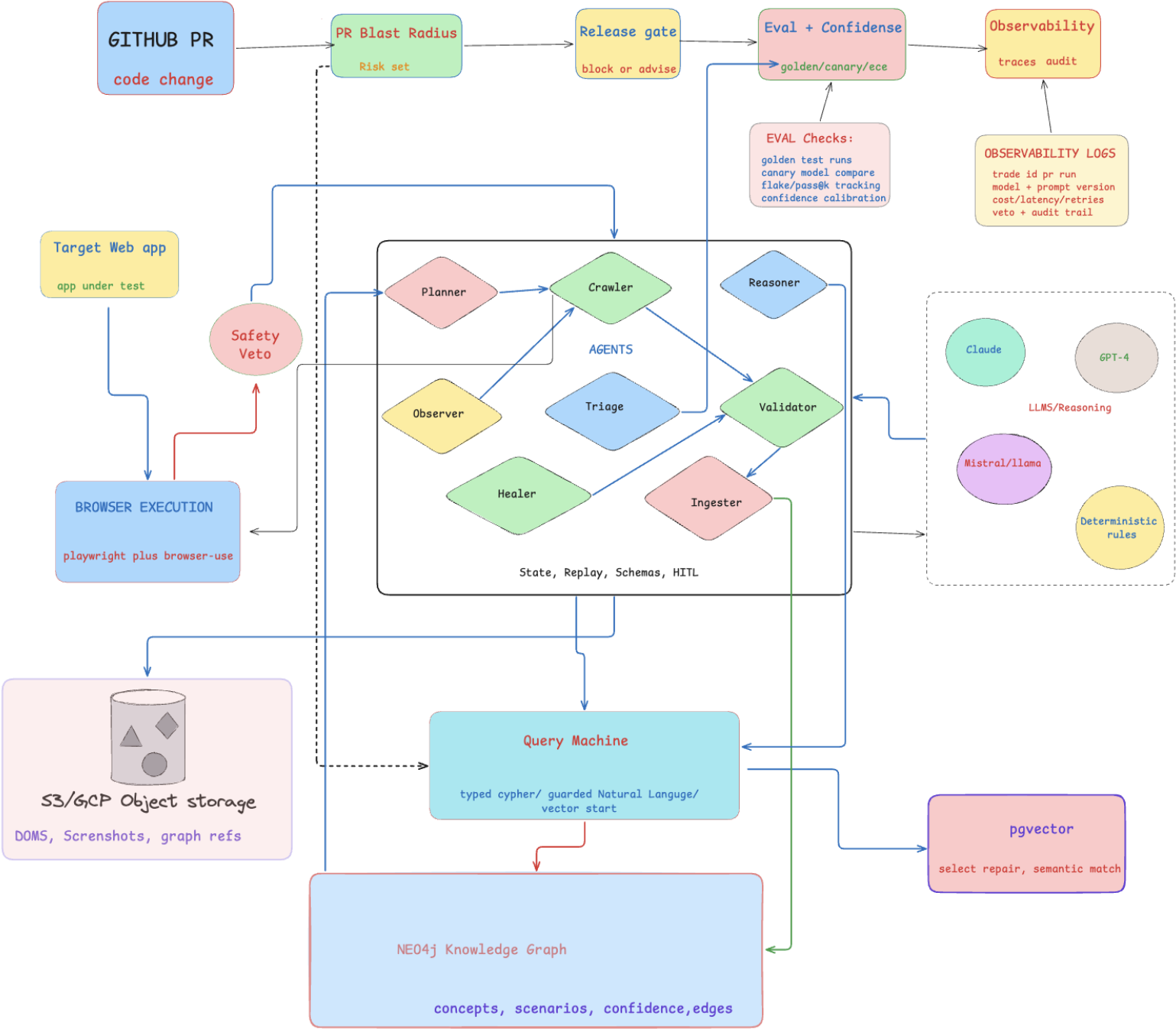
2. System design at a glance

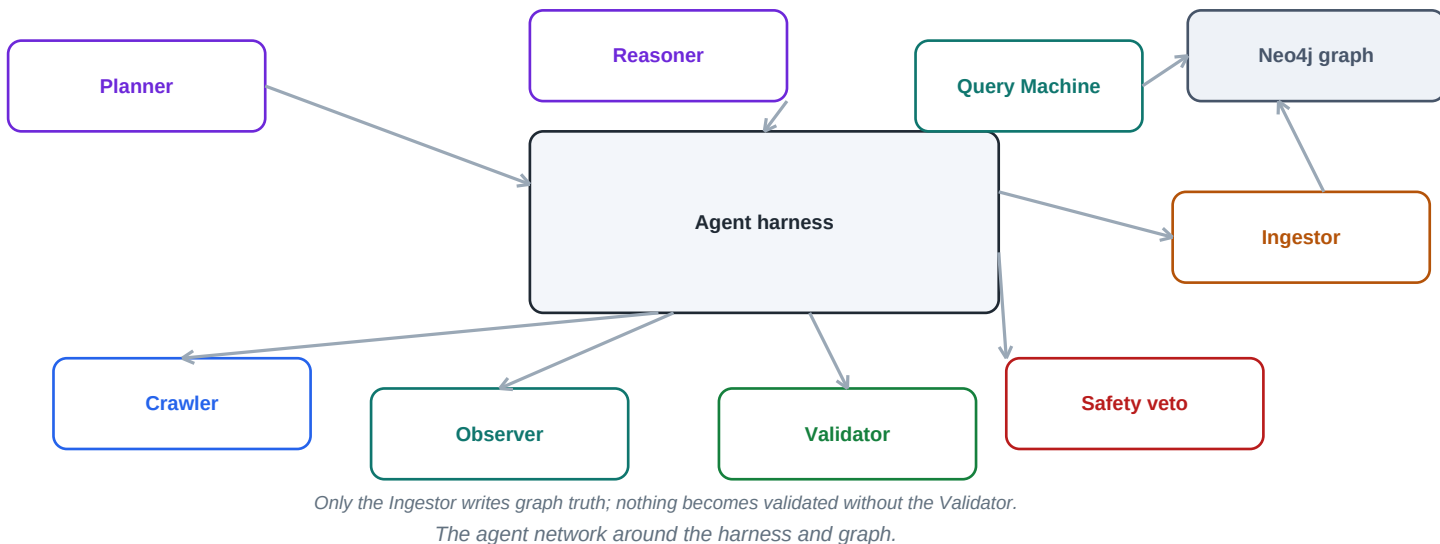
The full production system has three layers: cross-cutting release concerns, the agent runtime that acts on the app, and the memory layer that stores evidence and graph truth.

The release layer starts from code change and asks what is at risk. The runtime layer operates the app and validates what happened. The memory layer stores durable evidence, graph truth, and artifacts. Keeping these layers separate is what prevents a browser run from becoming an untraceable blob of model output.

Full production system design

Hand-drawn system design: release risk, agent harness, browser execution, graph memory, Query Machine, models, and observability.





3. The agent harness - this is the architecture

The harness owns state, retries, timeouts, tool dispatch, schema enforcement, deterministic replay, and human-in-the-loop interrupts. LangGraph can provide state-machine primitives; the durable value sits above it.

I call the harness the architecture because this is where agentic systems usually fail in production. The hard part is not making one model call. The hard part is knowing exactly what happened after twenty model calls, three browser actions, a retry, one safety veto, and a human login interruption.

- **State and replay:** durable Temporal workflows record inputs, outputs, model and prompt versions, trace IDs, and artifacts.
- **Structured dispatch:** agents return schema-validated objects; malformed output is a caught failure.
- **Retries and budgets:** latency, token, and circuit-breaker policies are observable, not silent spend.
- **HITL interrupts:** runs pause cleanly, ask a human, then resume with provenance intact.

This is why I would not rely on a bare autonomous-agent loop. The harness should make runs replayable. If a customer reports that a test blocked a release incorrectly, the system should show the exact inputs, outputs, artifacts, model versions, graph queries, and validation evidence that led to the decision.

4. Guardrails built into each agent

One shared safety layer across the whole system is the wrong shape because each agent is unsure about different things. Guardrails attach to each agent individually: every agent declares what not confident enough means, and what happens when it crosses that line.

The Part-A deny-list safety veto is the right shape: default-allow reversible exploration, hard-block enumerable irreversible actions such as purchase, payment, sign-out, and navigation away from the product under test.

The reason for keeping the veto separate from the crawler is simple: the crawler is allowed to misunderstand a page, but the safety layer is not. If the browser agent proposes a click on Place Order, the action should be blocked before the browser ever executes it. Similar hard boundaries apply to graph writes, confidence thresholds, and query execution.

Guardrail	What it does
Output validators / schema enforcement	The output must fit the contract or the step fails.
Confidence gating	Below threshold: escalate, fall back, or write unverified. No silent low-confidence passes.
Circuit breakers	Repeated failures trip a feature-level stop and surface a flag.
Fallback chains	Frontier model to small model to deterministic rule to human, per agent.
Deny-list veto	Irreversible actions blocked before execution by a separate process.

These guardrails are not meant to make exploration rigid. They make exploration safe. The system can still try unfamiliar reversible controls, but it cannot quietly cross a money boundary, write malformed graph data, or turn low confidence into a pass.

5. Model routing and composition

The job is choosing the right model for each task and combining them without one model's guess turning into the next one's fact. The principle is deterministic by default, small models for high-volume fuzzy work, and frontier models only where reasoning value is high and volume is low.

This matters for both cost and reliability. Browser execution creates many small decisions, so it should use cheaper, faster models where possible. Graph reasoning and root-cause analysis happen less often, so they can afford a stronger model. Validation and ingestion should be deterministic whenever the evidence is available.

Task	Route	Why
State resolution, validation, inference, ingestion	Deterministic code	About 90 percent of operations; must be explainable and repeatable.
Which element to click	Small / fast model	High volume, latency-sensitive UI skill.
Locator stability / flake classification	Fine-tuned small model	Narrow, repetitive, cheaper at scale.
Graph gap reasoning, root-cause chains, eval adjudication	Frontier model	Low volume, high reasoning value.

The table is a routing policy, not a vendor choice. The platform should be able to swap model providers as long as each route keeps the same contract: what input it receives, what output schema it must produce, how confidence is interpreted, and which deterministic checks follow it.

- Chain models without letting errors snowball: put deterministic checks or graph lookups between model stages.
- Pin prompt and model versions so provider changes are measured before customers file a ticket.
- Keep long sessions small by using the graph as long-term memory and querying only the needed slice.
- Keep frontier models off the hot path; cache small-model calls by graph-state signature and invalidate bounded subgraphs.

Cost shape

The exact provider prices will move, so I would track this with current vendor pricing in production. The architecture target is more important: small models do nearly all live crawl work, graph queries are deterministic, and frontier models run offline or on rare long-tail questions.

The cost story is also tied to the graph. If the graph already knows that a feature has not changed, the system should not rediscover it from scratch. If a PR touches one bounded area, the platform should invalidate that area instead of re-running the world. That is how the architecture avoids becoming an expensive crawler with memory bolted on afterward.

Unit	What runs	Budget / cost shape
Per feature crawl	About 25 small-model browser steps, roughly 75k input / 7.5k output tokens	Around cents, not dollars
Graph reasoning	Mostly cached deterministic Cypher; a few small calls near convergence	Near zero on unchanged graph state
Tier-1 graph query	Typed Cypher template	No model cost
Tier-2 graph query	Rare guarded NL-to-Cypher frontier call	Use for under 5 percent of graph questions
50 features nightly	1,500 crawls per customer per month	Keep in the low tens of dollars with caching
Offline frontier work	Rule discovery, eval adjudication, canaries	Amortised across customers

The spend stays controlled because unchanged pages are cached by graph-state signature, PRs invalidate bounded subgraphs instead of the whole app, and the frontier model never sits on the normal crawl path.

6. The production graph schema

I would keep the production graph smaller than it first appears. The graph does not need to model every DOM node or every visual component. It only needs to remember the things a release decision cares about: what feature was tested, what the browser saw, what action was attempted, what was actually proven, and which scenarios depend on those facts.

The most important simplification is this: the center of the graph is the **Concept**. A concept is a stable product meaning such as Add to Cart, Subtotal, Quantity Control, or Checkout Boundary. Selectors and page structure can change, but the meaning of Add to Cart survives a redesign. That is why I would model selectors as supporting evidence around a concept, not as the main unit of truth.

In a first production version, I would start with this smaller set of nodes. More detailed nodes can be added later, but these are enough to support missed-scenario reasoning, replay, PR blast radius, and validation history.

Node	Plain meaning	Why it exists
Tenant / App / Feature	Who owns the app, and which feature is under test.	Keeps one customer or feature from bleeding into another.
Run	One execution of the system.	Lets us compare what we knew in run N versus run N+1.
Observation	A page snapshot: URL, state, DOM/screenshot reference.	Lets us replay what the browser actually saw.
Concept	A meaningful behaviour or fact.	The stable spine of the graph: add_to_cart, subtotal, checkout_boundary.
Intent	An attempted action.	Records why an action ran and whether validation confirmed it.
Scenario	A behaviour worth testing.	Connects product behaviour to the concepts it depends on.
Selector / CodeArtifact	Locator and code references.	Used for selector repair and PR blast radius.

For implementation, I would still keep a more concrete property view. The simplified table explains the mental model; the table below is the production checklist I would hand to the person implementing the first graph schema.

Node	Key properties	Question answered
Tenant / App / Feature	ids	Scope one feature deeply without tenant bleed.
Run	commit_sha, started_at, model_version, cost	What did we know at run N versus N+1?
Observation	state, url, artifact_ref, screenshot_ref	Replay exactly what the browser saw.
Concept	key, kind, observed, validated, expected, confidence	Expected-not-observed; observed-not-validated.
Intent	source, status, risk, selector_ref, confidence	Why an action ran, passed, or was vetoed.
Scenario	key, status, confidence, last_confirmed_run	Which scenarios are trusted now versus decayed?
Selector	hash, role/css/xpath, stability, last_failed	Blast radius and self-healing.
CodeArtifact	path, symbol, commit_sha	Map PR change to selectors to concepts to scenarios.

I would not make the schema table carry every property. That is what made the earlier version hard to read. The useful properties are the obvious operational ones: ids for scoping, timestamps for history, status/confidence for trust, and artifact references for replay. Everything else can grow from real platform needs.

Edges are where the graph starts becoming useful. They tell us not just what exists, but how facts relate to each other. A scenario depends on concepts. An action targets a concept. One concept should cause another concept. That is what lets the platform find a missing test instead of only storing a crawl log.

Edge	Meaning
Run OBSERVED Observation	This run captured this page snapshot.
Observation SAW_CONCEPT Concept	This page showed this behaviour or fact.
Intent TARGETS Concept	This action tried to exercise this behaviour.
Scenario DEPENDS_ON Concept	This test scenario relies on this concept.

Edge	Meaning
Concept SHOULD_CAUSE Concept	This cause should produce this effect, such as Proceed to Checkout causing Checkout Boundary.
CodeArtifact / Selector BACKS Concept	This code or locator is tied to this product behaviour.

6.1 Why a graph and not just vectors

- **Absence:** missing tests are structural gaps, not similarity queries.
- **Temporal reasoning:** knowledge at commit X needs stamped edges and history.
- **Multi-hop traversal:** CodeArtifact to Selector to Concept to Scenario is the product.
- Vectors still help with selector repair and semantic matching, but only as an auxiliary index.

6.2 The few indexes I would start with

The index plan should also stay simple. I would not open with a long database-tuning section. I would index only the lookups the system performs constantly: finding a concept inside a feature, finding a run, finding a selector, and tracing from changed code to impacted scenarios.

Index	Why it matters
Concept by tenant + feature + key	Fast and correct concept lookup/upsert inside one feature.
Run by tenant + app + run_id	Replay and compare specific runs.
Selector by hash	Repair locators and connect selector changes to concepts.
CodeArtifact by commit/path/symbol	Start PR blast-radius traversal from changed code.
Causal edges by confidence/time	Ask what the graph believed at a point in time.

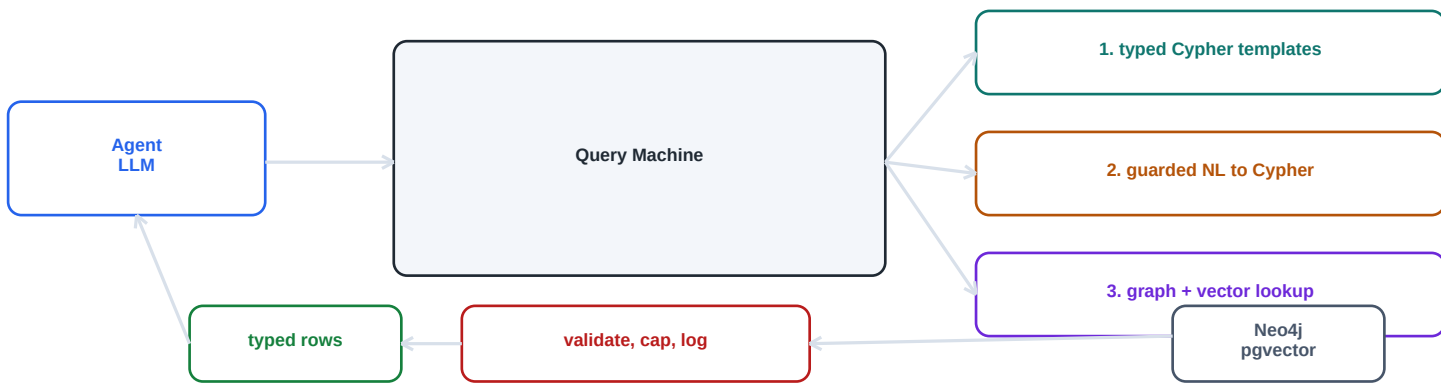
The production version of that index plan is slightly more explicit. I would still start with only the indexes below, then add more only after real query traces show pressure.

Index / constraint	Why it exists
Concept(tenant_id, feature_key, key)	Makes concept upserts correct and answers expected-vs-observed questions inside one feature.
Run(tenant_id, app_id, run_id)	Keeps run identity stable and replayable.
Selector(hash)	Fast selector repair and PR blast radius by locator content.
CodeArtifact(commit_sha, path, symbol)	Entry point for the PR hook.
Concept(tenant_id, feature_key)	Hot-path scoped feature lookups.
Run(commit_sha, started_at)	Temporal questions such as what changed between two commits.
SHOULD_CAUSE(valid_from, confidence)	Cheap queries for what the graph believed at time T and how sure it was.

7. The Query Machine

The Query Machine lives between agents and Neo4j. It turns 'what does the graph know?' into bounded, typed answers instead of dumping the graph into a prompt or letting an agent write arbitrary Cypher.

The simplest way to think about it is as a safe waiter between the agents and the graph database. An agent asks a structured question, the Query Machine decides the safest way to answer it, Neo4j returns rows, and the Query Machine gives the agent a typed response with provenance and confidence. The agent never needs direct unrestricted graph access.



Agents ask structured questions; the Query Machine returns typed rows with provenance and confidence.

Most questions should be Tier 1 because they are known product questions: missed scenarios, absence, blast radius, confidence, and history. Tier 2 exists for rare fuzzy questions, but it is guarded and read-only. Tier 3 uses vector search only to find the likely starting point; the graph still performs the reasoning.

- **Tier 1:** reviewed, parameterised graph-query tools such as `missed_scenarios`, `absence`, `blast_radius`, `neighbours`, `confidence_of`, and `what_did_we_know_at`.
- **Tier 2:** guarded natural-language to Cypher for the long tail: read-only, schema allow-listed, row/cost capped, logged, and reviewable.
- **Tier 3:** hybrid graph plus vector lookup: similarity finds the starting node, structure performs the reasoning.

8. Keeping the graph correct over time

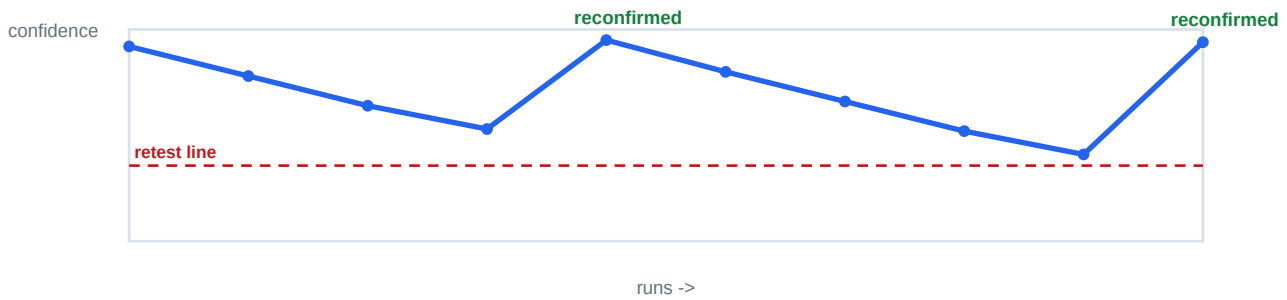
The graph is only useful if it stays fresh. Production applications change constantly: selectors move, pages are redesigned, models improve, and product behaviour drifts. The system therefore needs a clear freshness story. I would not rely on deleting and rebuilding the whole graph every night. That is expensive, noisy, and hides the reason something changed.

Instead, each crawl updates the evidence it touched, and each PR invalidates only the bounded area it can affect. If a selector repeatedly fails, the selector and dependent scenarios become stale. If a scenario is reconfirmed, confidence goes back up. This lets the platform explain both what it believes now and what it believed at an older commit.

Operation	Trigger	How
Incremental	Every crawl	Upsert observations, bump <code>last_seen</code> , append evidence, add <code>Scenario-[CONFIRMED_BY]->Run</code> .
Recomputed	A run touching a feature	Re-run inference and confidence for that feature only; never global.
Invalidated	PR touches mapped code; selector fails repeatedly; validation fails	Mark a bounded subgraph stale and push affected scenarios into the retest frontier.
Compounds	Many runs over time	Selector stability, flake rate, transition probabilities, and confidence-decay curves.

The table is the lifecycle in operational form: append evidence as runs happen, recompute only the feature that changed, invalidate bounded subgraphs when code or validation fails, and let confidence evolve over repeated runs.

- Conflict resolution preserves history instead of overwriting: old and new observations remain stamped by commit and time, so what the system believed at commit X stays answerable.
- Validated decisions feed the next run: confirmed scenarios raise priors; repeatedly flaky ones lower them.
- Full nightly recompute is rejected as the freshness story. Bounded invalidation is the defensible production shape.

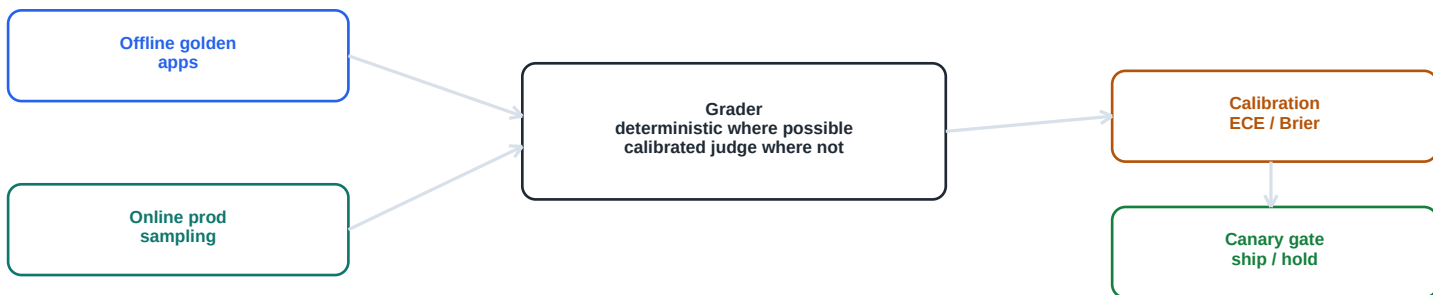


Confidence decays when runs do not reconfirm a scenario and snaps back on proof. Below the retest line it re-enters the frontier.

9. Eval and confidence

This is the layer that decides whether the system is a product or a science project. It protects against two confidently-wrong states: the agent clicked the wrong thing, and the graph inferred a scenario that is not real.

Eval is the part that keeps architecture honest. It is not enough for the system to generate convincing reports. It has to measure whether browser actions hit the right targets, whether validation actually predicts real success, whether graph-inferred scenarios reproduce, and whether confidence scores mean what they claim to mean.



Offline golden sets and online sampling feed graders; calibration and canary gates stand between a model change and customers.

The offline side gives the team stable examples that can be replayed after model or prompt changes. The online side samples real production runs to catch drift that golden apps miss. Canary gates sit between a new model and customers so a model upgrade cannot quietly make the crawler worse.

Question	Signal	Metric
Did it click the right target?	Trace plus positive/negative target check	Wrong-target rate, wander rate
Did the action succeed?	Deterministic post-condition after re-observe	Validation pass rate
Is the scenario reproducible?	Replay across clean sessions	Flake rate, pass@k
Are confidence scores honest?	Predicted confidence versus observed reproduction	ECE, Brier score
Did a model change regress us?	Canary on versioned golden trajectories	Delta versus baseline

These metrics are deliberately practical. Wrong-target and wander rate tell us if the crawler is acting on the wrong UI. Validation pass rate tells us if actions are actually being proven. Flake rate and pass@k tell us if scenarios are reproducible. ECE and Brier score tell us whether confidence is calibrated instead of decorative.

Use a managed eval store to move fast, but own the golden sets and grading logic in-house. LLM-as-judge must be calibrated against human spot-checks, and never used where a deterministic check exists.

10. Observability and operations

Observability is not just normal application logging here. A normal API service mostly needs request latency, error rate, and throughput. An agentic testing platform needs to explain a chain of decisions: why the planner picked a target, what the crawler clicked, what the safety layer allowed or blocked, what the validator accepted as evidence, and what finally entered the graph.

That means every run needs a single trace id that follows the whole path through the system. The trace should include the model version, prompt version, graph version, browser artifact references, validation result, confidence, cost, latency, retries, and any veto decisions. If a customer asks why a release was blocked, the platform should be able to replay the reasoning without asking anyone to read raw console logs.

The alerts should also be agent-specific. I would not only alert on 500s. I would alert when the crawler starts wandering more often, when validation failures spike after a model change, when many selectors become stale, when costs drift above budget, or when a graph query returns unusually low confidence. Those are the failures that matter in this product.

The rollout should be gradual. First the system observes and reports. Then it recommends which tests to run. Then it gates releases for low-risk features. Only after the evals and confidence calibration hold steady should it get closer to autonomous release decisions.

11. Multi-tenancy and scale

For a TestSigma-scale product, multi-tenancy is not an implementation detail. The graph, artifacts, sessions, screenshots, DOM captures, and customer vocabulary are all sensitive. I would scope every graph query by tenant, app, and feature from the beginning. For very large customers, I would consider a separate database or stronger physical isolation; for smaller tenants, strict logical partitioning may be enough.

The most important rule is that customer data should not become shared training data by accident. Selectors, DOM, screenshots, checkout flows, and private scenarios should stay inside the tenant boundary. The system can share reviewed structural ideas, such as 'checkout flows often have a cart boundary and a final purchase boundary,' but it should not copy one customer's implementation details into another customer's tests.

Area	Production stance
Graph	Always scoped by tenant/app/feature; larger customers can get stronger isolation.
Artifacts	DOM, screenshots, and videos live in per-tenant object storage with retention limits.
Models and prompts	Generic prompt templates can be shared; customer vocabulary and traces stay tenant-scoped.
Learning	Only reviewed structural priors cross tenants, never raw selectors or screenshots.

The same idea can be expressed as an isolation table: what is tenant-specific, and what can safely be shared as platform machinery.

Layer	Isolated per tenant	Shared
Graph	Tenant/app/feature partition; database-per-tenant for large accounts	Schema and inference engine code
Artifacts	Per-tenant namespace, retention, redaction	Storage implementation
Models / prompts	Customer vocabulary and trace versions	Generic templates
Learning	No selectors, DOM, screenshots, or scenarios cross tenants	Aggregated, reviewed structural priors only

The scale shape is straightforward: browser jobs run asynchronously on a queue, long workflows are durable, repeated graph states are cached, and Neo4j stores graph truth while object storage holds heavy artifacts. The Query Machine should bound traversals so one customer's large app cannot accidentally trigger an expensive global graph walk.

12. PR blast radius

The PR blast-radius hook is one of the clearest reasons to use a graph. When a pull request changes code, the system should not blindly say 'rerun every checkout test.' It should trace from the changed code to the affected selectors, from selectors to concepts, and from concepts to scenarios.

For example, if a PR touches the cart quantity component, the graph can find the quantity selector, the Change Quantity concept, the Subtotal concept, and the scenarios that depend on them. The release gate can then recommend a focused retest set: quantity updates subtotal, delete updates subtotal, and any checkout scenarios whose confidence depends on cart totals.

This is also where confidence matters. A scenario that was validated yesterday with strong evidence can be treated differently from a scenario that is already flaky or stale. The output should name the affected scenarios and explain why they are at risk, not just produce a flat list of tests.

13. Production concerns beyond the checklist

There are a few production concerns that are easy to understate in a prototype. The first is security. A browser testing system sees exactly the things companies are careful about: authenticated sessions, DOM content, screenshots, form fields, sometimes PII, and sometimes tokens. I would encrypt artifacts per tenant, redact sensitive values before storage, cap screenshot retention, and never log cookies or auth tokens.

The second concern is safe test data. The system can prove that it reached a checkout boundary, but it must not be able to place a real order. In production this means dedicated test accounts, seeded staging or sandbox data, and hard guardrails around final purchase actions. For third-party sites or production-like environments, the final purchase boundary should be structurally unreachable.

Authentication also needs a practical path. A human may handle OTP or CAPTCHA once, and the platform can then store an encrypted per-tenant browser session. But account-level actions such as sign-out, switch account, password changes, and payment settings should remain on the deny-list.

Finally, release gates should earn trust gradually. I would start in recommend mode, then move to blocking only for scenarios with strong validation history and acceptable flake rates. Known flaky tests should not blindly block a good release; they should be labeled, rerun, or escalated with context.

As a checklist, those production concerns become the following concrete stances.

Concern	Production stance
Security and privacy	DOM, screenshots, and sessions are sensitive. Encrypt per tenant, redact PII/tokens before storage, cap screenshot retention, and never log cookies or auth tokens.
Test data and safe environments	Use dedicated test accounts and seeded staging/sandbox data. The system can prove it reached a checkout boundary; it must not cross into real purchase.
Auth and sessions	A human handles OTP/CAPTCHA once; encrypted per-tenant sessions are refreshed safely. Account-level controls stay on the deny-list.
CI and release gates	Start in recommend mode, then earn blocking gates only after confidence and flake metrics hold. Known flaky tests should not block good releases blindly.

14. What I would refuse to ship

There are several things I would explicitly refuse to ship, because they create false confidence.

- **No path that can click final purchase.** Money pages must be structurally incapable of final submit. This should be red-teamed until it cannot buy.
- **No LLM self-report as validation.** If the action mutates state, the system needs an independent post-condition.
- **No uncalibrated confidence in the UI.** Confidence scores should be measured against replay or human-reviewed outcomes.
- **No nightly full-graph recompute as the freshness plan.** It is expensive and hides drift. The platform needs bounded invalidation.
- **No unreviewed cross-tenant learning.** Raw DOM, selectors, screenshots, and private scenarios must not cross tenant boundaries.
- **No hard-coded concept vocabulary as the final answer.** It is fine for Part A, but production needs tenant-aware concept adapters.

The refusal table is intentionally blunt. These are the places where I would rather slow the rollout than ship something that gives teams false safety.

Refusal	Why, and what must exist first
A path that can click final purchase	Money pages must be structurally incapable of final submit; red-team it until it cannot buy.
LLM self-report as validation	Every mutating action needs a deterministic post-condition.
Uncalibrated confidence in the UI	Confidence without calibration is false authority.
Nightly full-graph recompute as freshness	Expensive and hides drift; prove bounded invalidation on a golden tenant.

Refusal	Why, and what must exist first
Unreviewed cross-tenant learning	Only reviewed structural priors may cross tenants; never DOM, selectors, screenshots, or scenarios.
Hard-coded concept vocabulary as-is	Fine for one feature; production needs a learned per-tenant concept model behind an adapter.

The hardest problem is knowing the system is wrong before a human notices. The architecture answers with deterministic floors under the fuzzy parts, confidence that is measured rather than assumed, canaries between model changes and customers, and a graph that remembers what used to be true so drift is visible.

15. How Part A grounds every claim

Part B is intentionally larger than the prototype, but it is not detached from it. Part A already demonstrates the core loop in miniature. The browser-use executor is the crawler. The page observer captures evidence. The validator re-checks cart and checkout evidence instead of trusting the model. The graph stores observations and validated concepts. The reasoner then asks what was seen but never proven.

The production pieces are therefore extensions of working prototype ideas, not unrelated architecture boxes. The safety supervisor grows out of the current deny-list callback. The Query Machine grows out of deterministic graph queries such as missed scenarios and blast radius. The Ingestor grows out of the graph-store writes. Eval, observability, and multi-tenancy are the production shell around that loop.

Part-B idea	Part-A seed
Crawler acts, graph plans	GraphGuidedExplorer and BrowserUseIntentExecutor
Safety veto	safety_guard.veto_reason
Executor output is evidence, not truth	Re-observe plus cart/checkout assertions
Missed-scenario reasoning	infer_missed_scenarios and SHOULD_CAUSE
PR blast radius	pr_blast_radius.py and GraphStore.blast_radius

For a closer mapping from production claim to prototype code, I would keep this fuller traceability table as well.

Production claim	Prototype seed
Planner / Crawler split; graph plans, browser acts	GraphGuidedExplorer._autonomous_loop; BrowserUseIntentExecutor
Guardrails an LLM cannot bypass	safety_guard.veto_reason in the browser-use callback
Executor output is evidence, not truth	re-observe plus _run_assertions; cart-delta / checkout verification
Graph reasons about missed and causal gaps	infer_missed_scenarios; CAUSAL_EXPECTATIONS / SHOULD_CAUSE
Query Machine tier 1 and NL taste	infer_missed_scenarios; blast_radius; graph_expansion.expand_from_graph
Absence as graph query	seed_expected_concepts; missing_expected_concepts
PR blast radius traversal	pr_blast_radius.py; GraphStore.blast_radius
Provenance and confidence on writes	write_intent; write_observation

A concrete run makes the point clearer.

- **Discovered by the crawler:** Add to Cart works, and the quantity stepper moves. The Validator confirmed this by re-reading the actual cart.
- **Inferred by the graph:** Proceeding to checkout should reach the checkout boundary. The free crawl saw the control but never proved the effect.
- **Why the graph inferred it:** action.proceed_to_checkout SHOULD_CAUSE domain.checkout_boundary. Observed cause, expected effect, no proof.
- **What happened next:** the graph turned that gap into a directed probe, the browser clicked Proceed, and the run reached the real checkout boundary.

16. The stack, and why

The stack does not need to be exotic. I would start with common tools and keep the boundaries clear. Playwright and browser-use operate the browser. Neo4j stores graph truth. Object storage holds heavy artifacts. A small service layer exposes the Query Machine. Temporal or an equivalent workflow system gives durable runs and replay.

The main thing to avoid is turning the whole system into one free-form autonomous agent. The durable value is the contract between components: browser execution is sandboxed, graph writes are typed, validation is independent, and model usage is routed by risk and cost.

Layer	Start with
Agent harness	Custom harness over LangGraph-style primitives; Temporal for durable runs.
Browser execution	Playwright plus browser-use, sandboxed behind an action contract.
Graph store	Neo4j with provenance, confidence, and temporal history.
Retrieval	pgvector or Weaviate only as auxiliary semantic lookup.
Query Machine	Parameterized Cypher tools first; guarded NL-to-Cypher for rare questions.
Models	Small executors for high-volume UI work; frontier model for rare reasoning/judging.
Eval and observability	Golden apps, canaries, trace ids, cost/latency budgets, decision audit trails.
Artifacts	S3/GCS for screenshots, DOM, and video; graph stores references only.

The fuller stack table below keeps the tradeoff explicit: what I would start with, and what I would avoid because it hides risk.

Layer	Start with	Avoid
Agent harness	Custom harness over LangGraph primitives; Temporal for durable runs	A fully autonomous agent with no state/replay contract
Browser execution	Playwright plus browser-use, sandboxed	No action contract or veto
Graph store	Neo4j with temporal provenance and confidence on edges	Graph-shaped dependencies hidden in relational tables
Retrieval	pgvector / Weaviate as auxiliary hybrid lookup	Vector similarity replacing graph truth
Query Machine	Parameterised Cypher tools plus guarded NL-to-Cypher	Free-form model-written Cypher on the hot path
Models	Frontier reasoning/judge; small executors; fine-tuned narrow classifiers; deterministic validation	Provider name as the whole strategy
Eval	Versioned golden apps, managed eval store, calibrated judge, canaries	Shipping prompt changes without canaries
Observability	OpenTelemetry, trace IDs, decision audit	Plain logs with no trace IDs
Async / queue	Celery + Redis, FastAPI, Docker/K8s, AWS/GCP	Fire-and-forget cron for retryable crawls
Artifacts	S3/GCS; Neo4j holds refs only	Large DOM/screenshot blobs inside Neo4j

In one line: the valuable part is not the clever crawler. It is the loop between something that explores and something that remembers, with a hard wall between what the system did and what it is allowed to believe.